

UsLab-Eng-01 — Access & Security Strategy

Access Strategy — UsLab-Eng-01 (DRAFT, separate from the build)

Not part of the initial cluster build. This is a short, standalone proposal for how we give developers and users access to the cluster and its apps, and how we keep it secure. Review and decide before we implement any of it.

1. Who needs what

Audience	Needs	Should NOT have
Platform/ops (you)	Full cluster admin, node SSH, Helm	—
Developers	Deploy + debug in <i>their own</i> namespaces, read logs, port-forward	Cluster-admin, node SSH, other teams' namespaces
End users	Only the apps over HTTPS (*.antbrains.com)	Any kube API / kubectl access
CI/CD	Deploy to specific namespaces via a token	Interactive/standing human credentials

Principle: **least privilege, per namespace, no shared logins, everything auditable.**

2. Two planes of access — keep them separate

A. Cluster API access (kubectl) — for ops + developers

- **Don't hand out the RKE2 admin kubeconfig.** That's cluster-admin for everyone.
- Give each person their **own identity** and a **scoped RBAC** binding:
 - Per-team **namespace(s)** with a **RoleBinding** to a **developer** Role (get/list/watch/create/update/delete on workloads, configmaps, secrets, pods/log, pods/portforward — within that namespace only).
 - Read-only **ClusterRole** (view) only if they genuinely need cluster-wide read.
- **Identity options (pick one):**
 1. **Rancher Authentication** (simplest with our stack) — Rancher is already planned (plan §11). Wire it to **GitHub/Google/OIDC**, manage users + project/namespace roles in Rancher, hand out Rancher-issued kubeconfigs.
 2. **OIDC directly on the API server** (kube-apiserver **--oidc-*** flags via RKE2 config) against Google/Entra/Keycloak — more setup, no Rancher dependency.
- **Never** create long-lived ServiceAccount tokens for humans. Humans = OIDC.

B. Application access (HTTPS) — for end users

- Apps are reached only through **Nginx Proxy Manager** at `*.antbrains.com` (TLS via Let's Encrypt). The kube API is not exposed to users.
- Put auth **in front of** apps that need it, so each app doesn't reinvent login:
 - NPM **Access Lists** (basic-auth + IP allowlist) for quick internal-only apps.
 - For real SSO, add **oauth2-proxy** (or Authelia/Authentik) as a small in-cluster service and have NPM proxy through it → one login for many apps.
- Internal-only tools (Rancher UI, NPM admin :81, dashboards): keep them on the **LAN/VPN only** — do not give them public DNS + cert.

3. Securing it — baseline controls

Layer	Control
Identity	SSO/OIDC, MFA enforced at the IdP. No shared accounts. Named CI tokens, rotated.
RBAC	Default-deny; per-namespace Roles; review bindings quarterly. No blanket cluster-admin.
Network	Default-deny NetworkPolicies per namespace (Calico). Only NPM namespace exposed north-south. Admin UIs LAN/VPN-only.
Secrets	Don't commit secrets to git. Use Sealed Secrets / External Secrets; restrict <code>get secrets</code> by RBAC.
Workload	Pod Security Admission <code>restricted</code> on app namespaces; drop caps; runAsNonRoot; read-only rootfs where possible.
Edge	Force HTTPS in NPM, HSTS, modern TLS only. Rate-limit/allowlist admin endpoints.
Nodes	SSH by key only, ops group only, ideally via a bastion/VPN. No password auth.
Audit	Enable kube-apiserver audit logging; ship to the monitoring stack (plan §12). Alert on RBAC changes & failed auth.
Backups	VolSync (Direct) + app-level dumps for stateful apps; test restores.

4. Suggested rollout (when we do this)

1. Stand up **Rancher** + wire **OIDC** (GitHub/Google). Define Projects = teams.
2. Create per-team **namespaces** + `developer` Role + RoleBindings via Rancher.
3. Apply **default-deny NetworkPolicies** + **PSA restricted** to app namespaces.
4. Add **oauth2-proxy** for app SSO; convert NPM hosts that need login to use it.
5. Lock down admin surfaces (NPM :81, Rancher, dashboards) to **VPN/LAN**.
6. Turn on **audit logging** + alerts; document the **token rotation** runbook.

5. Open questions for you

- Which IdP — **GitHub, Google Workspace, or Entra/Keycloak**?
- Is there a **VPN/bastion** for admin surfaces, or do we need to add one?
- Do end users need **SSO** (oauth2-proxy) or is per-app basic-auth enough initially?
- Team → namespace mapping: what are the teams?

Once you pick directions here, I'll turn this into concrete manifests (RBAC Roles/Bindings, NetworkPolicies, oauth2-proxy, audit config) as a follow-up — kept separate from the cluster build.